Федеральное государственное автономное образовательное учреждение
высшего образования
«Самарский национальный исследовательский университет
имени академика С.П. Королёва»

Факультет информатики
Кафедра технической кибернетики

ПОДРОБНЫЙ РАЗБОР

лабораторной работы №3
по дисциплине «Анализ социальных сетей»

Обработка графов и визуализация

Васильев Артём, группа 6231-010402D

Самара, 2026

Содержание

1. Задача и итог работы 3
2. Архитектура решения 4
3. Задание №1: сбор данных 4
3.1. Как работает запрос . . . 4
3.2. Постраничная загрузка . . . 4
3.3. Обработка ошибок . . . 5
3.4. Сохранение прогресса . . . 5
4. Основы теории графов 5
4.1. Метрики графа . . . 6
5. Как устроена библиотека NetworkX 6
5.1. Как граф хранится в памяти . . . 6
5.2. Доступ к данным . . . 7
5.3. Подграфы: представление, а не копия . . . 7
5.4. Метрики и алгоритмы . . . 7
5.5. Сохранение в GraphML . . . 8
5.6. Укладка spring_layout . . . 9
5.7. Метод Лувена . . . 9
5.8. Рисование . . . 10
6. Задание №2: логика построения графа 10
6.1. Какие вершины и рёбра входят в граф . . . 10
6.2. Как считаются связи . . . 11
6.3. Разбор примера . . . 11
6.4. Почему метрики считаются дважды . . . 12
7. Задание №3: логика визуализации 12
7.1. Выделение сообществ . . . 12
7.2. Укладка . . . 13
7.3. Размер вершин . . . 14
7.4. Толщина рёбер и слои . . . 14
7.5. Цвета и легенда . . . 15
8. Результаты и их интерпретация 15
8.1. Ограничения . . . 15
8.2. Выполнение требований задания . . . 15
9. Как запустить и что можно менять 16
10. Вопросы для самопроверки 16

1. Задача и итог работы

Лабораторная работа состоит из трёх заданий: собрать списки друзей пользователя VK, построить по ним граф и визуализировать его с выделением сообществ. Этот документ написан как разбор для самостоятельного изучения: он объясняет не только что делает код, но и почему он устроен именно так, а основное внимание уделено библиотеке NetworkX.

Таблица 1 – Итоговые результаты

Показатель	Значение
Друзей исходного пользователя	171
Недоступных профилей	95 (56%)
Вершин / рёбер графа	172 / 1241
Крупных сообществ	3 (61, 55 и 45 человек)
Модулярность разбиения	0,637
Рёбер между друзьями внутри трёх сообществ	1032 из 1070 (96%)

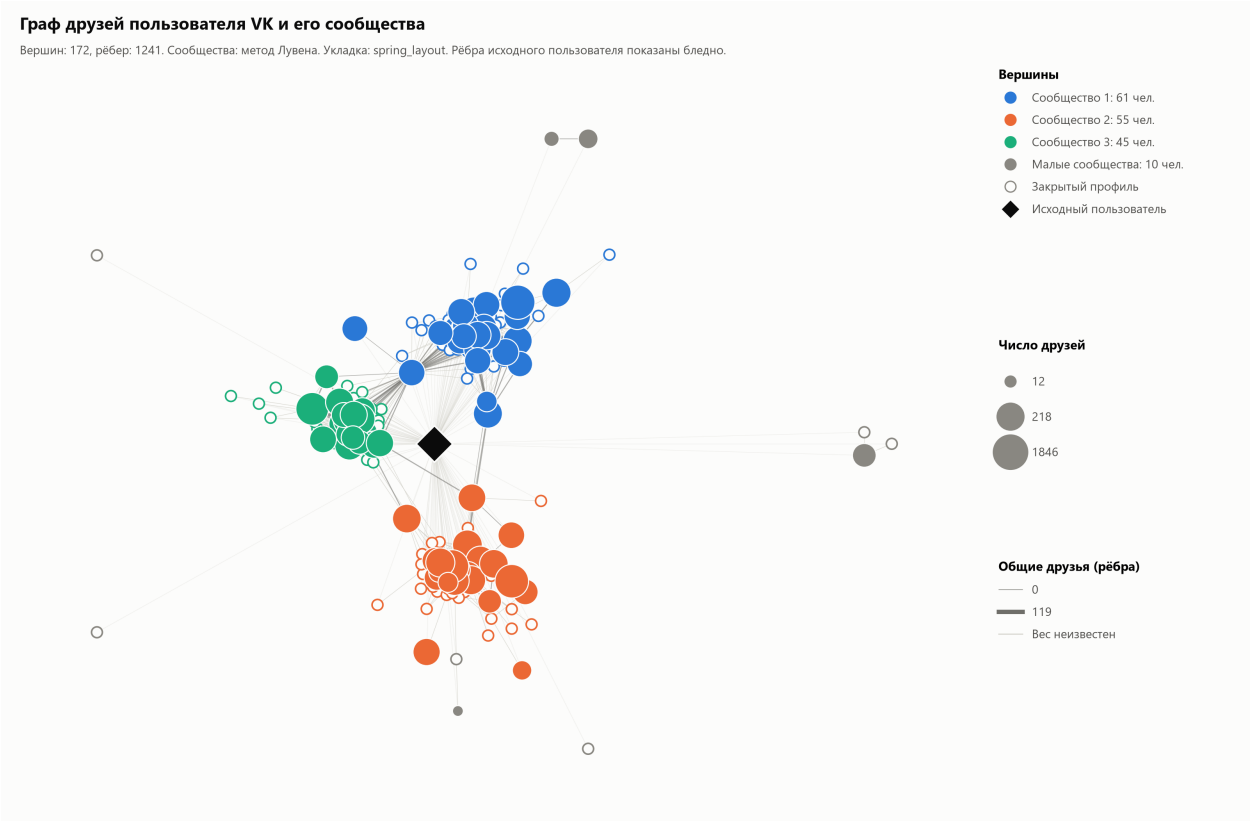


Рисунок 1 – Итоговый граф друзей

На рисунке видны три плотные группы, соединённые небольшим числом рёбер. Именно такую структуру и должен находить анализ сообществ. Дальше разберём, как получен этот результат.

2. Архитектура решения

Решение разбито на три класса, по одному на задание. Каждый этап записывает результат в файл, а следующий читает его. Благодаря этому этапы независимы: чтобы перерисовать граф, не нужно заново обращаться к VK, а сбор, занимающий минуты, выполняется один раз.

Таблица 2 – Этапы обработки

Этап	Класс и модуль	Вход	Выход
1. Сбор	VKFriendsCollector collect_friends.py	API VK, токен	friends_<id>.json
2. Граф	FriendsGraphBuilder build_graph.py	friends_<id>.json	graph_<id>.graphml graph_<id>_metrics.json
3. Рисунок	FriendsGraphVisualizer visualize_graph.py	graph_<id>.graphml	friends_graph.png graph_<id>_communities.json

Запуск всей цепочки выполняет `main.py` через функцию `run_lab3_pipeline` из `pipeline.py`. Первым шагом при этом всегда идёт короткое обращение к VK: сбор определяет числовой ID пользователя. Если файл уже собран, запросов о друзьях повторно не будет.

3. Задание №1: сбор данных

3.1. Как работает запрос

Метод `friends.get` возвращает число друзей и список их ID. Библиотека vk_api превращает методы API в обычные функции Python: вызов `vk.friends.get(user_id=123)` отправляет HTTP-запрос с токеном и возвращает словарь вида `{"count": 2, "items": [456, 789]}`. Если VK вернул ошибку, библиотека вызывает исключение `ApiError` с её кодом в поле `code`.

Сбор идёт на двух уровнях: сначала друзья исходного пользователя, затем друзья каждого из них. Второй уровень нужен, потому что только из него видны связи между самими друзьями.

3.2. Постраничная загрузка

За один запрос метод возвращает не более 5000 ID. Функция `get_friends` запрашивает следующие части, пока не наберёт весь список. Сдвигом служит число уже полученных друзей:

```
while True:
    response = self.request_friends(
        user_id=user_id, count=MAX_FRIENDS_BY_REQUEST, offset=len(friends)
    )
    items = response.get("items", [])
    friends.extend(items)
    if not items or len(friends) >= response.get("count", 0):
        return friends
    time.sleep(REQUEST_PAUSE)
```

3.3. Обработка ошибок

Главная идея этой части в том, что разные ошибки значат разное, и обрабатывать их одинаково нельзя.

Таблица 3 – Обработка ошибок VK API

Код	Смысл	Действие	Почему
15, 18, 30	Нет доступа, страница удалена, профиль закрыт	Пустой список, продолжить	Данных действительно нет
6, 29	Превышен лимит запросов	Ждать и повторить, максимум 3 раза	Данные есть, нужно только подождать
Другие	Например, неверный токен	Сохранить прогресс и остановиться	Проблема не в конкретном человеке

Главная ловушка. Если при превышении лимита записать пустой список, у человека ложно исчезнут все друзья. Программа при этом не упадёт, а граф окажется неверным, и заметить это будет трудно. Такую же ошибку дала бы запись пустых списков при неверном токене, поэтому в этом случае сбор останавливается.

3.4. Сохранение прогресса

Прогресс сохраняется каждые 10 друзей, а блок `try ... finally` сохраняет его и при любом прерывании. При повторном запуске уже обработанные друзья пропускаются. Запись идёт сначала во временный файл, затем он заменяет основной функцией `os.replace`: если сбой случится во время записи, прежний файл останется целым.

```
{
    "user_id": 123,
    "friends": [456, 789],
    "friends_of_friends": {"456": [123, 789, 111], "789": []},
    "unavailable": [789]
}
```

В JSON ключи всегда строки, поэтому `456` сохраняется как `"456"`. Поле `unavailable` отличает закрытый профиль от человека, у которого действительно нет друзей: в обоих случаях список пуст.

Настройки приватности. На практике сбор сначала упал с ошибкой 30: профиль был закрыт для этого запроса. Решение: открыть профиль в настройках VK, как описано в комментарии в начале `collect_friends.py`. Настройки других людей изменить нельзя, поэтому 95 из 171 профилей остались недоступны.

4. Основы теории графов

Прежде чем переходить к библиотеке, зафиксируем понятия, которые считает код.

Таблица 4 – Основные понятия

Понятие	Определение	В этой работе
Вершина	Объект в графе	Пользователь VK
Ребро	Связь двух вершин	Дружба
Вес ребра	Число, приписанное связи	Число общих друзей
Неориентированный граф	У рёбер нет направления	Дружба в VK взаимна
Степень вершины	Число её рёбер	Число друзей внутри сети
Путь	Цепочка вершин, соединённых рёбрами	Цепочка знакомств
Компонента связности	Часть графа, где из любой вершины можно дойти до любой другой	Связанная группа людей
Изолированная вершина	Вершина без рёбер	Друг без видимых связей

4.1. Метрики графа

Плотность показывает, какая доля всех возможных рёбер существует, где n — число вершин, а m — число рёбер:

$$density = 2m / (n \cdot (n - 1))$$

Коэффициент кластеризации вершины показывает, какая доля пар её соседей связана между собой, то есть насколько часто «друзья друга тоже дружат». Для вершины со степенью k и числом рёбер между соседями T:

$$C = 2T / (k \cdot (k - 1))$$

Средний коэффициент — это среднее по всем вершинам. Вершины со степенью 0 и 1 дают значение 0.

Эксцентриситет вершины — расстояние до самой дальней от неё вершины. **Диаметр** графа — максимальный эксцентриситет, то есть длина самого длинного из кратчайших путей.

5. Как устроена библиотека NetworkX

NetworkX — библиотека Python для работы с графами. В работе используется версия 3.7. Все примеры ниже запущены на этой версии, а вывод приведён настоящий. Описание внутреннего устройства проверено по исходному коду установленной библиотеки.

5.1. Как граф хранится в памяти

Класс `nx.Graph` хранит граф как **вложенные словари**. Это называется «словарь смежности»: для каждой вершины записаны её соседи, а для каждого соседа — атрибуты ребра.

```
G = nx.Graph()
G.add_node("Анна", friends_count=4)
G.add_edge("Анна", "Борис", weight=2)
G.add_edge("Анна", "Вера", weight=1)
G.add_edge("Борис", "Вера", weight=1)
```

```
G.add_edge("Вера", "Глеб", weight=0)
G.add_node("Дарья")

# Содержимое G.adj: вершина -> сосед -> атрибуты ребра
"Анна":  {"Борис": {"weight": 2}, "Вера": {"weight": 1}}
"Борис": {"Анна": {"weight": 2}, "Вера": {"weight": 1}}
"Вера":  {"Анна": {"weight": 1}, "Борис": {"weight": 1}, "Глеб": {"weight": 0}}
"Глеб":  {"Вера": {"weight": 0}}
"Дарья": {}
```

Из этого устройства следуют важные свойства:

- **Быстрые проверки.** Проверка соседства и доступ к ребру в среднем не зависят от размера графа, потому что это поиск в словаре.
- **Каждое ребро записано дважды.** Оно есть у обеих вершин, но атрибуты общие: проверка `G["Анна"]["Борис"] is G["Борис"]["Анна"]` даёт True. Поэтому изменение атрибута видно с обеих сторон.
- **Вершиной может быть любой хешируемый объект:** число или строка. У нас это числовой ID.
- **Вершины создаются автоматически.** `add_edge` сам добавляет отсутствующие вершины, поэтому Борису и Глебу отдельный `add_node` не понадобился.
- **Повторное добавление не дублирует.** Граф `nx.Graph` не допускает кратных рёбер: повторный вызов только обновит атрибуты.

5.2. Доступ к данным

Таблица 5 – Основные способы доступа (вывод на примере выше)

Выражение	Результат	Что возвращает
`G.nodes["Анна"]`	{'friends_count': 4}	атрибуты вершины
`G.edges["Анна", "Борис"]`	{'weight': 2}	атрибуты ребра
`G.degree("Вера")`	3	степень вершины
`G.number_of_nodes()`	5	число вершин
`G.number_of_edges()`	4	число рёбер
`G.nodes(data=True)`	пары (вершина, атрибуты)	используется при переборе
`G.edges(data=True)`	тройки (u, v, атрибуты)	используется при переборе

5.3. Подграфы: представление, а не копия

Метод `G.subgraph(nodes)` не копирует данные, а создаёт **представление**: объект, который смотрит на исходный граф и показывает только указанные вершины и рёбра между ними. Это быстро и не расходует память, но такое представление нельзя изменять. Попытка добавить в него вершину даёт ошибку: «Frozen graph can't be modified». Для изменений нужен `.copy()`.

В коде подграфы используются дважды: граф без исходного пользователя и наибольшая компонента связности.

5.4. Метрики и алгоритмы

Таблица 6 – Функции NetworkX, использованные в работе

Функция	Что считает	Пример выше
`nx.density(G)`	плотность: 2·4 / (5·4)	0,4
`nx.clustering(G)`	коэффициент каждой вершины	Вера: 1/3
`nx.average_clustering(G)`	среднее по всем вершинам	0,467
`nx.connected_components(G)`	генератор множеств вершин	2 компоненты
`nx.number_of_isolates(G)`	число вершин степени 0	1
`nx.diameter(G)`	наибольший эксцентриситет	ошибка, см. ниже

У Веры три соседа, то есть три пары, но связана только одна (Анна и Борис), поэтому её коэффициент равен 1/3. Компоненты связности находятся обходом графа: начиная с непосещённой вершины, алгоритм собирает всё, что из неё достижимо.

Функция `nx.diameter` по умолчанию вычисляет эксцентриситет каждой вершины поиском кратчайших путей и берёт максимум. Для несвязного графа расстояние бесконечно, и функция сообщает: «Found infinite path length because the graph is not connected». Поэтому в коде диаметр считается только по наибольшей компоненте. Для примера он равен 2.

5.5. Сохранение в GraphML

GraphML — текстовый формат на основе XML, его понимают и другие программы, например Gephi. В начале файла объявлены атрибуты и их типы, затем перечислены вершины и рёбра:

```
<key id="d3" for="edge" attr.name="weight" attr.type="long" />
<key id="d0" for="node" attr.name="friends_count" attr.type="long" />
...
<graph edgedefault="undirected">
  <node id="101">
    <data key="d0">204</data>
    ...
  </node>
  <edge source="101" target="102">
    <data key="d3">0</data>
    ...
  </edge>
</graph>
```

С этим форматом связаны две особенности, которые повлияли на код:

- **ID становятся строками.** В XML всё хранится как текст, и по умолчанию `read_graphml` возвращает вершины как строки ('101', '102'). Поэтому при чтении указан `node_type=int`.
- **Значение None не поддерживается.** Попытка записать его даёт ошибку: «GraphML writer does not support <class 'NoneType'> as data values.». Поэтому неизвестное число друзей хранится как 0, а отдельный атрибут `available` показывает, что значение неизвестно.

5.6. Укладка spring_layout

У графа нет координат, их вычисляет алгоритм укладки. Функция `spring_layout` реализует алгоритм Фрюхтермана–Рейнгольда. Его идея — физическая модель: все вершины отталкиваются друг от друга, как одноимённые заряды, а рёбра притягивают соединённые вершины, как пружины. Система постепенно приходит в равновесие.

Для вершин на расстоянии d с идеальным расстоянием k силы равны:

$$\text{отталкивание} = k^2 / d, \qquad \text{притяжение} = A \cdot d^2 / k$$

Здесь A — элемент матрицы смежности: вес ребра или 0, если ребра нет. Ход работы алгоритма:

1. Вершины получают случайные начальные координаты, поэтому от `seed` зависит результат.
2. На каждой итерации для всех вершин считается суммарная сила.
3. Вершины сдвигаются по направлению силы, но не дальше текущей «температуры» t.
4. Температура снижается с каждой итерацией, поэтому сдвиги уменьшаются и рисунок стабилизируется.
5. Алгоритм останавливается после `iterations` шагов или раньше, если сдвиги стали меньше `threshold`.
6. Координаты масштабируются к диапазону `scale`.

Таблица 7 – Параметры spring_layout

Параметр	По умолчанию	В коде	Зачем
k	$1/\sqrt{n} \approx 0{,}08$	0,35	вершины дальше друг от друга
iterations	50	200	более устойчивое равновесие
weight	'weight'	None	рёбра с весом 0 не должны терять притяжение
seed	None	42	повторяемый результат

Значение по умолчанию для k указано для основной компоненты из 163 вершин. Для графов менее чем из 500 вершин используется точный расчёт сил для всех пар, для больших — приближённый метод на разреженной матрице.

Почему нельзя передавать веса. Из формулы видно: при A = 0 притяжения нет. У рёбер закрытых профилей вес записан как 0, поэтому с весами они потеряли бы притяжение, хотя связь существует. Это и есть причина `weight=None`.

5.7. Метод Лувена

Метод Лувена ищет разбиение вершин на сообщества, максимизируя **модулярность**. Она показывает, насколько внутри сообществ больше рёбер, чем было бы при случайном соединении вершин с теми же степенями:

$$Q = \Sigma_c \left[L_c / m - \gamma \cdot (k_c / 2m)^2 \right]$$

Здесь L_c — число рёбер внутри сообщества c, k_c — сумма степеней его вершин, m — число рёбер графа, γ — параметр разрешения (по умолчанию 1). Первое слагаемое — доля рёбер внутри сообщества, второе — ожидаемая доля при случайном графе.

Пример. Возьмём две группы по 4 человека, где внутри каждой все дружат, а между группами одно ребро: 8 вершин и 13 рёбер. Для каждой группы L = 6 и k = 13, поэтому:

$$Q = 2 \cdot [\ 6/13 - (13/26)^2\ ] \approx 0{,}423$$

Запуск `louvain_communities` действительно разделяет эти группы и даёт Q = 0,423. Если объединить всех в одно сообщество, получится Q = 0,0.

Алгоритм повторяет две фазы:

1. **Локальные перемещения.** Сначала каждая вершина — отдельное сообщество. Вершины в случайном порядке переходят к сообществу соседа, если это увеличивает Q, пока перемещения дают выигрыш.
2. **Объединение.** Каждое сообщество сжимается в одну вершину, а рёбра между ними суммируются. На таком графе снова выполняется фаза 1.

Алгоритм останавливается, когда рост Q становится меньше `threshold` (по умолчанию 10^{-7}). Порядок перебора случайный, поэтому результат зависит от `seed`. Функция возвращает список множеств вершин. Значения Q выше примерно 0,3 обычно считают признаком заметной структуры.

5.8. Рисование

Функции `draw_networkx_nodes` и `draw_networkx_edges` рисуют граф через matplotlib по заранее вычисленным координатам. Они принимают списки вершин или рёбер и значения стиля для каждого элемента, поэтому их можно вызывать по частям. Порядок вызовов задаёт порядок слоёв: сначала рёбра, затем вершины поверх них. Размер `node_size` задаётся как площадь в квадратных пунктах, поэтому в легенде маркер имеет диаметр √size.

6. Задание №2: логика построения графа

6.1. Какие вершины и рёбра входят в граф

Вершины — исходный пользователь и его друзья, всего 172. Друзья друзей вершинами не становятся: их списки нужны только для поиска связей и подсчёта общих друзей. Иначе граф вырос бы до десятков тысяч вершин.

Ребро ставится только при настоящей дружбе. В задании допускается другое толкование: ребро при наличии любых общих знакомых. Но тогда граф стал бы очень плотным, а ребро означало бы не дружбу, а «есть общий знакомый». Выбрано первое, поскольку в результатах работы задание описывает рёбра как дружеские связи.

6.2. Как считаются связи

Списки друзей хранятся в виде множеств (`set`), потому что проверка принадлежности и пересечение в них работают быстро. Перебор всех пар выполняет `itertools.combinations`: при 171 друзьях это 14535 пар.

```
for friend_id in self.friends:
    self.add_edge(self.source_id, friend_id)

for user_a, user_b in combinations(self.friends, 2):
    if self.are_friends(user_a, user_b):
        self.add_edge(user_a, user_b)
```

Таблица 8 – Решения и их причины

Решение	Проблема, которую оно решает
Дружба проверяется с обеих сторон	Закрытый профиль виден в списке открытого: иначе настоящие связи потерялись бы
Исходный пользователь исключён из пересечения	Он есть в списке каждого друга: иначе все веса были бы завышены на 1
Атрибут weight_known	Вес 0 у закрытого профиля значит «неизвестно», а не «нет общих друзей»
Атрибут available	friends_count = 0 у закрытого профиля значит «неизвестно»
Отказ работать с незавершённым сбором	Несобранные списки выглядели бы как пустые

6.3. Разбор примера

Пусть у вас четыре друга, у Глеба закрытый профиль, а X — знакомый вне сети.

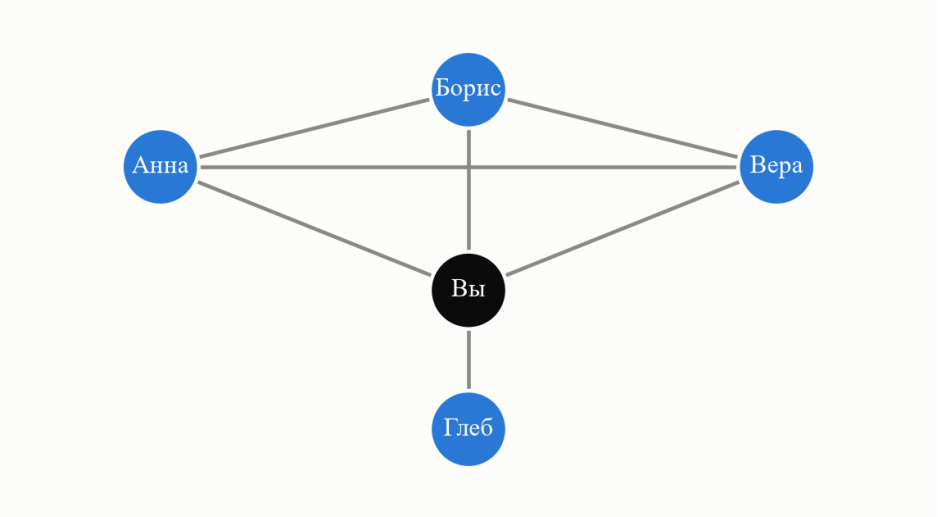


Рисунок 2 – Пример: граф из пяти вершин

Таблица 9 – Расчёт весов для примера

Пара	Пересечение списков	Вес	Вес известен
Анна — Борис	{Вы, Вера, X}, Вы исключён	2	да
Анна — Вера	{Вы, Борис}, Вы исключён	1	да
Вы — Анна	{Борис, Вера}	2	да
Вы — Глеб	список Глеба пуст	0	нет

Здесь: Анна — {Вы, Борис, Вера, X}, Борис — {Вы, Анна, Вера, X}, Вера — {Вы, Анна, Борис}, Глеб — закрыт.

6.4. Почему метрики считаются дважды

Исходный пользователь связан со всеми, что искажает метрики полного графа. Любые двое соединены через него, поэтому диаметр не больше 2. Любая дружба двух друзей образует с ним треугольник, поэтому кластеризация завышена. Поэтому для описания сети важнее граф без него.

Таблица 10 – Метрики графа

Метрика	Полный граф	Без исходного
Вершин	172	171
Рёбер	1241	1070
Плотность	0,084	0,074
Средний коэффициент кластеризации	0,638	0,447
Компонент связности	1	6
Изолированных вершин	0	3
Вершин в наибольшей компоненте	172	163
Диаметр наибольшей компоненты	2	7

Без исходного пользователя 163 из 171 друзей остаются связаны между собой, а самый дальний путь равен 7. Коэффициент 0,447 при плотности 0,074 показывает, что друзья одного человека дружат между собой намного чаще, чем можно было бы ожидать по общей плотности.

Степень внутри сети: медиана 11, максимум 54. У пяти самых связанных друзей 54, 42, 39, 38, 36 связей.

7. Задание №3: логика визуализации

7.1. Выделение сообществ

Перед окончательным выбором я сравнил три варианта запуска метода Лувена на настоящем графе.

Таблица 11 – Варианты выделения сообществ

Вариант	Сообществ	Крупнейшие	Одиночек	Q
Без исходного, без весов	9	61, 55, 45	3	0,637
Без исходного, с весами	105	25, 23, 20	101	0,635
С исходным, без весов	4	56, 44, 42, 30	0	0,559

Выбран первый вариант. С весами все 95 закрытых профилей стали сообществами из одного человека: для алгоритма вес 0 равносилен отсутствию связи. Модулярность при этом почти не изменилась, поэтому на одно это значение полагаться нельзя. С исходным пользователем Q ниже: он связан со всеми и размывает границы.

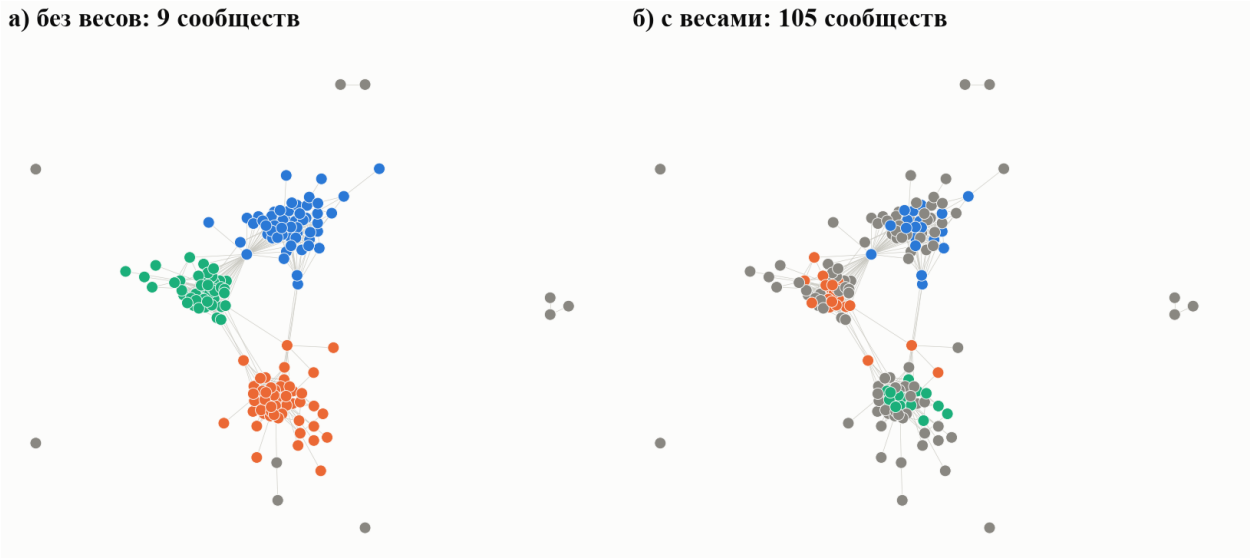


Рисунок 3 – Влияние весов на сообщества: серым показаны малые сообщества, цвета на двух частях рисунка не соответствуют друг другу

Первый вариант устойчив: при 20 разных значениях `seed` разбиения почти совпадают (средний ARI 0,99).

7.2. Укладка

Первая попытка с `spring_layout` по всему графу дала плохой результат. Несвязанные вершины отталкиваются, а пружин, которые их удержат, нет, поэтому они разлетаются. После масштабирования основная часть сети сжалась в маленькое пятно.

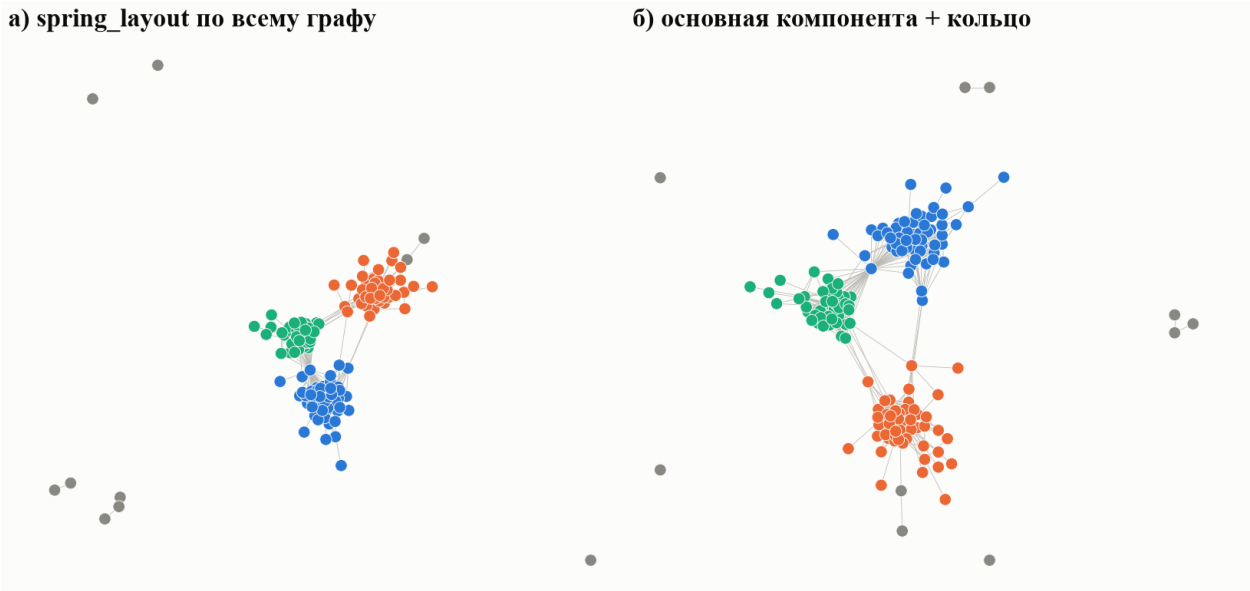


Рисунок 4 – Укладка до и после исправления

Теперь укладка считается только для наибольшей компоненты. Малые компоненты (5 шт., всего 8 вершин) расставляются по кольцу вокруг неё, а исходный пользователь ставится в центр. Расстояние до вершин на кольце ничего не значит.

```
small_components = components[1:]
for number, component in enumerate(small_components):
    angle = 2 * np.pi * number / len(small_components)
    center = OUTER_RING_RADIUS * np.array([np.cos(angle), np.sin(angle)])
```

7.3. Размер вершин

Значения приводятся к диапазону от 100 до 1000 нормализацией:

$$\text{size} = \text{low} + (\text{high} - \text{low}) \cdot (x - x_{min}) / (x_{max} - x_{min})$$

Но у открытых профилей число друзей от 12 до 1846 при медиане 221. Обычная нормализация сделала бы почти все вершины мелкими, а одну огромной. Поэтому сначала берётся логарифм `np.log1p`, который сжимает большие значения. Для сравнения: при линейной шкале медиана 221 получает размер около 203 из 1000.

Закрытым профилям назначается минимальный размер, а сами они рисуются полыми, чтобы «неизвестно» не выглядело как «мало друзей».

7.4. Толщина рёбер и слои

Среди рёбер между друзьями вес известен только у 295 из 1070. Поэтому рёбра разделены на три группы, рисуемые по порядку: рёбра исходного пользователя (171, очень бледные), рёбра с неизвестным весом (775, тонкие светлые) и рёбра с известным весом (295, толщина от 0,5 до 4 по весу).

7.5. Цвета и легенда

Различить девять цветов трудно, а шесть из девяти сообществ содержат всего 10 человек. Поэтому три крупных сообщества получают цвет, а остальные — серый. Легенд три: вершины, размер и толщина. На рисунке нет осей, потому что координаты укладки не имеют смысла, и нет подписей людей: на рисунок не попадают ни ID, ни имена.

8. Результаты и их интерпретация

Таблица 12 – Сообщества (первые шесть; остальные три состоят из одного закрытого профиля)

Сообщество	Человек	Закрытых	Рёбер внутри	Плотность
1	61	34	360	0,197
2	55	30	355	0,239
3	45	25	317	0,320
4	3	2	2	0,667
5	2	0	1	1,000
6	2	1	1	1,000

- **Структура выражена хорошо:** Q = 0,637, а 96% рёбер между друзьями лежат внутри трёх сообществ.
- **Смысл групп по данным определить нельзя.** У нас есть только ID, поэтому что объединяет каждую группу (например, школа, университет, работа), может сказать только человек, знающий эту сеть.

8.1. Ограничения

- **Закрытые профили.** 95 из 171 профилей недоступны.
- **Непроверяемые пары.** Связь двух закрытых профилей увидеть нельзя. Таких пар 4465 из 14535 (31%). Поэтому плотность и кластеризация относятся только к видимой части сети.
- **Веса.** Вес известен только у 371 из 1241 рёбер (30%).
- **Изолированные вершины.** Человек может быть изолирован только потому, что его связи скрыты.
- **Сообщества закрытых профилей.** Закрытый профиль попадает в группу только по связям с открытыми, поэтому его принадлежность определена менее надёжно.

8.2. Выполнение требований задания

Таблица 13 – Сопоставление с результатами из задания

Требование	Где выполнено
Файл со списками друзей	friends_<id>.json
Файл графа	graph_<id>.graphml
Рисунок: размер, толщина, цвет по сообществам	friends_graph.png
Описание характеристик графа	graph_<id>_metrics.json, graph_<id>_communities.json и разделы 6–8

9. Как запустить и что можно менять

```
python main.py                    # вся цепочка, короткое обращение к VK
python -m lab3.build_graph        # только граф, без VK
python -m lab3.visualize_graph    # только рисунок, без VK
```

Таблица 14 – Настройки

Что изменить	Где
Исходный пользователь	поле self.user в collect_friends.py или аргумент collect(user)
Паузы и повторы запросов	REQUEST_PAUSE, RETRY_PAUSE, MAX_RETRIES
Расстояние между вершинами	LAYOUT_SPREAD в visualize_graph.py
Размер вершин	MIN_NODE_SIZE, MAX_NODE_SIZE
Толщина рёбер	MIN_EDGE_WIDTH, MAX_EDGE_WIDTH
Порог для цветного сообщества	MIN_COLORED_COMMUNITY

Файлы с ID исключены из git, а окно с рисунком при запуске нужно закрыть, чтобы программа завершилась. Пути в `pipeline.py` содержат конкретный ID, поэтому при смене пользователя их нужно обновить.

10. Вопросы для самопроверки

Почему друзья друзей не вершины?
Граф вырос бы до десятков тысяч вершин. Их списки нужны только для связей и весов.

Почему дружба проверяется с обеих сторон?
Если один профиль закрыт, связь видна только в списке открытого.

Почему исходный пользователь не входит в вес?
Он есть в списке каждого своего друга, поэтому завысил бы все веса на 1.

Почему диаметр полного графа равен 2?
Исходный пользователь связан со всеми, поэтому любые двое соединены через него.

Что произойдёт, если вызвать diameter на несвязном графе?
Будет ошибка: расстояние между компонентами бесконечно. Поэтому он считается по наибольшей компоненте.

Почему Лувен запущен без весов?
Вес 0 у закрытых профилей значит «неизвестно», а с весами алгоритм считает такие связи отсутствующими, и все закрытые профили становятся одиночками.

Что показывает модулярность?
Насколько внутри сообществ больше рёбер, чем при случайном соединении вершин с теми же степенями.

Как работает spring_layout?
Все вершины отталкиваются, рёбра притягивают, а сдвиги уменьшаются с «температурой», пока система не придёт в равновесие.

Почему размер вершин логарифмический?
Разброс от 12 до 1846: при линейной шкале почти все вершины были бы минимальными.

Зачем node_type=int при чтении GraphML?
В XML всё хранится текстом, и иначе ID вершин станут строками.